AL-MAJARRAT

PROJECT DOCUMENTATION

Evidence-Grounded Quran and Hadith Research Assistant

A local-first retrieval-augmented generation system

Prepared by: [Student / Project Team]

Guided by: [Supervisor / Guide]

Institution: [Institution Name]

Department: [Department Name]

Academic year: 2026

Abstract

Al-Majarrat is a locally operated research assistant for exploring Quranic verses and Hadith passages. It combines a structured SQLite corpus, SQLite FTS5 lexical search, locally generated text embeddings, rank fusion, and an optional local large language model (LLM). A command-line interface supports direct search and question answering; a browser interface exposes question entry, source filters, retrieved references, answer copying, locally retained conversation history, and a responsive sidebar. The system is designed to ground generated answers in retrieved source passages, attach reference tags, and present source text for user verification.

The implemented data store contains 34,157 Hadith records from six collections and 56,124 Quran rows representing Arabic text and translation records. The current vector tables contain 34,157 Hadith embeddings and 6,236 Quran verse embeddings. These are database coverage counts, not measures of answer accuracy. The project health test suite contains four tests; all four passed in the validation run recorded for this report. Data-file and SQLite schema checks also passed. Ollama-dependent validation was intentionally skipped during that run, and no independent retrieval-quality benchmark or user study is claimed.

This document describes the implemented scope, architecture, data preparation, retrieval and generation workflow, verified results, limitations, operation, and future work. Al-Majarrat is a research aid, not a religious authority; users should verify translations, references, and interpretations against the cited source editions and qualified scholarship.

Keywords: Quran, Hadith, SQLite FTS5, semantic search, embeddings, retrieval-augmented generation, Ollama, local AI.

Contents

Abstract 2
Contents 3
Chapter 1 — Introduction 5
1.1 Background of the Study 5
1.2 Problem Definition 5
1.3 Objectives of the Project 5
1.4 Scope of the Project 5
1.5 Motivation 5
Chapter 2 — Literature Review 7
2.1 Existing Approaches / Related Work 7
2.1.1 Full-Text Search 7
2.1.2 Dense Semantic Retrieval 7
2.1.3 Retrieval-Augmented Generation 7
2.2 Limitations of Existing Approaches 7
2.3 Research Gap 7
2.4 Proposed Approach 7
Chapter 3 — System Analysis 8
3.1 System Requirements 8
3.1.1 Hardware Requirements 8
3.1.2 Software Requirements 8
3.1.3 Functional Requirements 8
3.1.4 Non-Functional Requirements 8
3.2 Feasibility Study 9
3.2.1 Technical Feasibility 9
3.2.2 Economic Feasibility 9
3.2.3 Operational Feasibility 9
3.2.4 Schedule Feasibility 9
3.3 Methodology Used 9
Chapter 4 — System Design 10
4.1 System Architecture 10
4.2 Modules and Data Flow 10
4.3 Database Design 10
4.4 Use Cases and Sequence 11

4.4.1 Primary Actor and Use Cases 11
4.4.2 Answer Request Sequence 11
Chapter 5 — Implementation 12
5.1 Technology Stack 12
5.2 Algorithms and Pseudocode 12
5.2.1 Corpus Construction 12
5.2.2 Embedding Indexing 12
5.2.3 Hybrid Retrieval 12
5.2.4 Grounded Answer Generation 12
5.3 Implementation Steps 13
5.4 Project Folder Structure 13
5.5 Operational User Interfaces 13
5.6 User Interface Screenshots 13
Chapter 6 — Results and Discussion 15
6.1 Verified Database Snapshot 15
6.2 Functional Verification 15
6.3 Example Input and Expected System Path 15
6.4 Analysis of Results 16
Chapter 7 — Conclusion and Future Scope 17
7.1 Summary of Work 17
7.2 Achievements 17
7.3 Limitations 17
7.4 Future Enhancements 17
Chapter 8 — References 18
Chapter 9 — Appendix 19
9.1 Database Structure Summary 19
9.2 Local API Summary 19
9.3 Validation and Test Evidence 19
9.4 Main Project Files 19
9.5 User Operation Quick Reference 19

The table of contents is generated from the document headings. In Word, right-click it and choose Update Field → Update entire table if you edit the report.

Chapter 1 — Introduction

1.1 Background of the Study

The Quran and Hadith are foundational sources for Islamic study. Readers often need to locate material by a verse reference, topic, wording, collection, narrator, or translation. Large text collections make manual lookup slow, while general-purpose language models can produce unsupported statements or uncertain citations if they answer without a source-grounding step.

Al-Majarrat addresses this information-retrieval problem by maintaining local Quran and Hadith corpora and combining lexical and semantic retrieval. Search results are formatted with references and source text. When the optional local LLM is available, retrieved passages are supplied as constrained context for a natural-language response. The system is intended to make exploratory research more accessible while keeping supporting material visible.

1.2 Problem Definition

A user asking a thematic question may not know the exact wording or reference needed to find relevant source material. Keyword-only search can miss semantically related phrasing; unconstrained generation can return claims that are not supported by a verifiable source. There is a need for a searchable, locally operable system that can retrieve relevant Quran and Hadith passages, expose their references, and distinguish source evidence from generated explanation.

1.3 Objectives of the Project

- Create a project-root-stable data and source layout for the application.
- Prepare Quran and Hadith source files and consolidate them into SQLite.
- Provide full-text search using SQLite FTS5 and semantic retrieval using locally generated embeddings.
- Combine lexical and semantic rankings to produce relevant passages.
- Provide CLI and browser workflows for searching and asking questions.
- Constrain local LLM answers to retrieved context and display references for verification.
- Provide health checks and tests for paths, source data, and database schema.

1.4 Scope of the Project

The implemented application covers English-language question handling, Quran and Hadith retrieval, selected English translations with Uthmani Arabic text, a local Ollama-compatible generation endpoint, and a local browser UI. Quran queries include direct verse-key and Surah handling. Hadith data spans Sahih al-Bukhari, Sahih Muslim, Sunan an-Nasa'i, Sunan Abi Dawud, Jami' at-Tirmidhi, and Sunan Ibn Majah.

The project does not implement user accounts, a hosted multi-user service, scholarly adjudication, or a validated fatwa workflow. Citations do not prove that generated answers are correct. Fine-tuning dataset export and a QLoRA training script are included as development utilities; the default serving path uses the configured Ollama chat model, and a trained adapter is not automatically loaded.

1.5 Motivation

The project pairs convenient thematic exploration with visible source evidence. A local-first approach permits operation without sending each research query to a hosted LLM provider, subject to the installed local model

and hardware. The aim is assistance in locating and reviewing sources—not replacing primary texts, established scholarship, or qualified guidance.

Chapter 2 — Literature Review

2.1 Existing Approaches / Related Work

Relevant technical approaches include database text search, term-frequency ranking, dense vector retrieval, and retrieval-augmented generation. They are complementary: lexical search is effective for exact words and names; embedding search can surface related concepts with different wording; grounded generation can summarize retrieved passages conversationally.

2.1.1 Full-Text Search

SQLite FTS5 provides an embedded full-text index and relevance ranking. It suits a local application because it operates alongside the project database without a separate search server.

2.1.2 Dense Semantic Retrieval

An embedding model maps text to numeric vectors. Query/document similarity can retrieve passages whose wording differs from the query. Al-Majarrat uses the configured nomic-embed-text Ollama model and cosine similarity over stored JSON vectors.

2.1.3 Retrieval-Augmented Generation

RAG systems retrieve supporting documents and provide them as context to a language model. Here, a prompt instructs the model to answer from supplied Quran/Hadith context, retain exact reference tags, include English translation text, and state when no direct references were found.

2.2 Limitations of Existing Approaches

Keyword search depends on token overlap and can be sensitive to spelling and phrasing. Dense retrieval depends on embedding-model coverage, corpus quality, and ranking. Generative models may still misread, overgeneralize, or misattribute supplied passages. Combining methods does not prove relevance or scholarly soundness.

2.3 Research Gap

A practical gap exists between simple reference lookup and conversational answers traceable to source passages. A local application should support direct lookup and thematic search, keep source text visible, and avoid depending on a hosted model. Al-Majarrat implements these features; systematic relevance, factuality, and usability evaluations remain future work.

2.4 Proposed Approach

Prepare the source corpora, build a SQLite database with FTS5 tables, generate and store embeddings, retrieve lexical and semantic candidate sets, combine rankings using Reciprocal Rank Fusion (RRF), and optionally submit selected evidence to a local chat model. CLI and web UI share the RAG module.

Chapter 3 — System Analysis

3.1 System Requirements

3.1.1 Hardware Requirements

The repository declares no formal minimum hardware. Runtime requirements depend on selected local models; inference can require substantial memory and may benefit from a compatible GPU. Retrieval-only use avoids the chat model, but semantic retrieval requires the embedding endpoint unless it falls back to lexical-only search.

3.1.2 Software Requirements

Component	Technology
Runtime	Python; verified environment: Python 3.14.7.
Database	SQLite with FTS5 virtual tables.
Embedding service	Ollama; configured model nomic-embed-text.
Generation service	Ollama OpenAI-compatible endpoint; qwen2.5:3b.
Browser UI	HTML, CSS, JavaScript; Python standard-library HTTP server.
Training utilities	PyTorch, Transformers, PEFT, Accelerate, bitsandbytes.
Testing	Python unittest; four project health tests.

3.1.3 Functional Requirements

ID	Requirement	Status / evidence
FR-01	Search Hadith from CLI and web-backed retrieval.	Implemented.
FR-02	Search Quran text and direct references.	Implemented.
FR-03	Retrieve using lexical and semantic signals.	FTS5, vectors, cosine, RRF.
FR-04	Generate answers through a local LLM.	Implemented; service required.
FR-05	Filter all/Quran/Hadith in web UI.	Implemented.
FR-06	Display retrieved source passages and references.	Implemented.
FR-07	Retain/reopen completed browser conversations.	localStorage.
FR-08	Delete an individual conversation.	Implemented.
FR-09	Report basic data/backend readiness.	Validator and status API.

3.1.4 Non-Functional Requirements

- Portability: configured paths resolve from the project root.
- Traceability: results and prompt instructions retain references and evidence.
- Local operation: web server defaults to loopback; model endpoints are local.
- Input bounds: web questions are limited to 2,000 characters; bodies to 16 KB.
- Maintainability: code, data, tests, and UI assets are separated.
- Reliability: health checks validate required files and database tables.

3.2 Feasibility Study

3.2.1 Technical Feasibility

The project uses Python, SQLite/FTS5, local HTTP model APIs, and browser technologies. A populated database, vector indexes, and passing health tests demonstrate that principal data components work in the checked environment. End-to-end generation depends on Ollama and configured models.

3.2.2 Economic Feasibility

Local models can avoid per-query hosted LLM fees. Costs include hardware, storage, electricity, initial downloads, and maintenance. No formal cost study has been conducted.

3.2.3 Operational Feasibility

The CLI supports direct search and interactive questions; the web app provides a browser workflow. Setup requires corpus/database preparation and Ollama for full semantic/generative operation. Users should verify AI responses against displayed sources.

3.2.4 Schedule Feasibility

The repository contains extraction, indexing, application, training, and validation stages. No schedule, effort estimate, or milestone record was found in the reviewed implementation evidence; no schedule-completion claim is made.

3.3 Methodology Used

The staged workflow is: obtain data, normalize records, build search indexes, create vectors, retrieve and fuse candidates, compose grounded prompts, and present results through CLI or web UI. Automated health checks cover project structure and schema; this report distinguishes these from evaluations not yet performed.

Chapter 4 — System Design

4.1 System Architecture

Al-Majarrat is a single-machine client/server-style application. The Python web server serves static UI assets and ask/status APIs. The web handler and CLI call the shared RAG module. SQLite stores source records, full-text indexes, and serialized vectors. Ollama exposes local embedding and chat APIs.

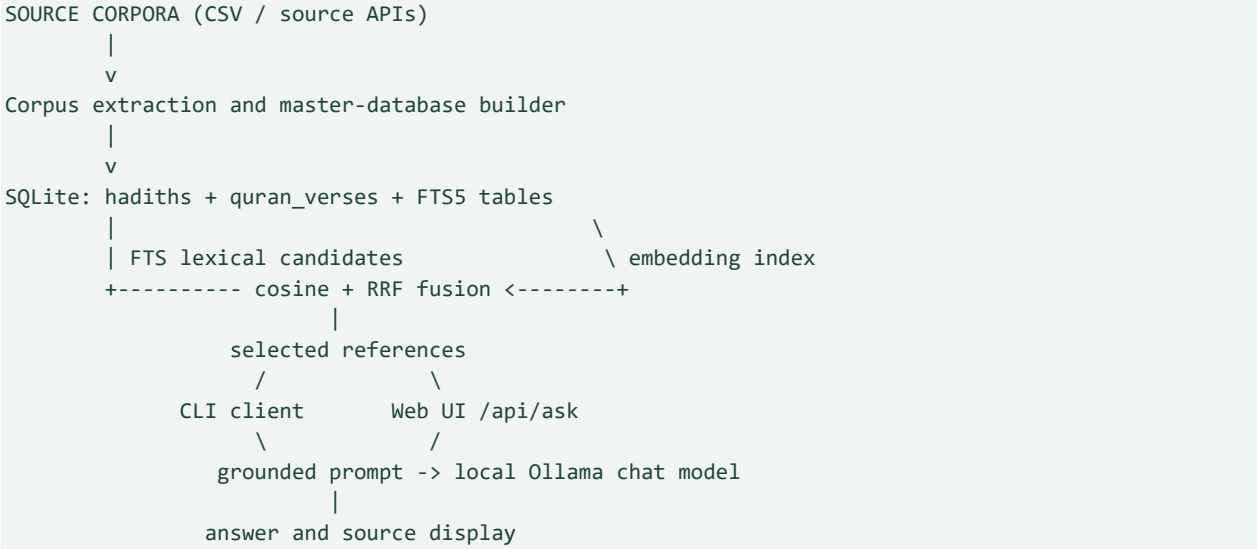


Figure 4.1 — Logical architecture of implemented components.

4.2 Modules and Data Flow

Module	Responsibility
src/extract_*.py	Acquire/transform Quran and six Hadith collections.
src/create_master_corpus.py	Build source tables, indexes, and FTS5 tables.
src/build_vector_index.py	Generate embeddings in batches and persist vectors.
src/al_majarrat.py	Resolve references, retrieve/fuse sources, prompt, call Ollama, CLI.
src/search_majarrat.py	Interactive and command-line lexical search.
src/serve_web_ui.py + web/index.html	Serve UI/API, validate requests, report status, render results.
src/validate_project.py + tests/	Health checks and project tests.
export/train scripts	Create fine-tuning data and optional QLoRA adapter; separate from default serving.

4.3 Database Design

Table	Key fields / role
hadiths	id; collection/book/chapter; narrator; Arabic/English text; grading; reference.
quran_verses	id; chapter/verse; verse_key; Uthmani text; translation resource/name/author/text.
hadiths_fts	FTS5 external-content index over collection, reference, narrator, and texts.
quran_fts	FTS5 external-content index over verse key, Arabic and translation text.

hadith_vector_index	hadith_db_id unique key, reference, serialized embedding JSON.
quran_vector_index	verse_key unique key and embedding JSON for preferred English translation.

4.4 Use Cases and Sequence

4.4.1 Primary Actor and Use Cases

The primary actor is a local researcher. Use cases include searching text or a reference, selecting Quran/Hadith/all, reviewing sources, copying answers, reopening saved history, and deleting an individual conversation. The app does not implement accounts or login.

4.4.2 Answer Request Sequence

(1) User submits question and mode. (2) UI sends bounded JSON to /api/ask. (3) Server validates the request and invokes shared retrieval. (4) Retrieval obtains an embedding when available, searches FTS/direct references, ranks vectors, fuses candidates, and returns evidence. (5) Server builds a constrained prompt and requests local chat completion. (6) API returns answer and sources. (7) Browser renders and stores the completed conversation locally.

Chapter 5 — Implementation

5.1 Technology Stack

Layer	Technology / details
Application	Python and standard library HTTP, SQLite, JSON, HTTP client.
Data store	SQLite: data/db/al_majarrat_master.db.
Lexical index	SQLite FTS5 external-content tables.
Vector index	JSON-serialized embeddings; cosine similarity in application code.
Embedding model	nomic-embed-text served by Ollama.
Chat model	qwen2.5:3b at Ollama OpenAI-compatible endpoint.
Frontend	Static HTML/CSS/JavaScript; responsive UI and localStorage.
Training	Transformers/PEFT QLoRA utilities; CUDA PyTorch needed.

5.2 Algorithms and Pseudocode

5.2.1 Corpus Construction

```
FOR each of six Hadith CSV inputs:
  map collection metadata; skip rows without Arabic and English text
  append normalized records to master Hadith CSV
CREATE SQLite source tables and indexes
LOAD Hadith and Quran CSV rows
CREATE and populate FTS5 external-content tables
```

5.2.2 Embedding Indexing

```
SELECT eligible Quran translations and English Hadith texts
SKIP records already indexed
FOR each batch (up to 64 texts): request local embeddings
  verify vector count; store JSON; commit progress
REPORT indexed coverage
```

5.2.3 Hybrid Retrieval

```
query_vector = request_embedding(question)  # may be unavailable
resolve direct Quran verse / Surah reference when present
collect FTS5 candidates and top cosine-similarity candidates
RRF(candidate) = sum(1 / (60 + rank)) over ranked lists
return up to 6 Quran passages and 5 Hadith passages
format references and English text for the prompt
```

The implementation also includes query normalization and synonym/fuzzy support. Direct verse queries take precedence for Quran selection. These retrieval heuristics have not received a full quality evaluation.

5.2.4 Grounded Answer Generation

```
system prompt = English + exact citations + no fabrication
context = selected passages or explicit no-match statements
POST prompt to configured local chat endpoint
return answer or explicit offline/error status with retrieved context
```

5.3 Implementation Steps

1. Create or activate a Python virtual environment and install requirements.txt.
2. Ensure required corpus CSV files exist under data/corpora.
3. Run the master corpus builder. It deletes and recreates the configured DB; back up data before rebuilding.
4. Install/start Ollama and pull nomic-embed-text and qwen2.5:3b.
5. Run the vector index builder; it skips already indexed records and reports coverage.
6. Run validation and tests.
7. Launch the CLI or web UI and open the local address printed by the server.

```
python -m pip install -r requirements.txt
python src/create_master_corpus.py
ollama pull nomic-embed-text
ollama pull qwen2.5:3b
python src/build_vector_index.py
python src/validate_project.py --skip-ollama
python src/serve_web_ui.py
```

5.4 Project Folder Structure

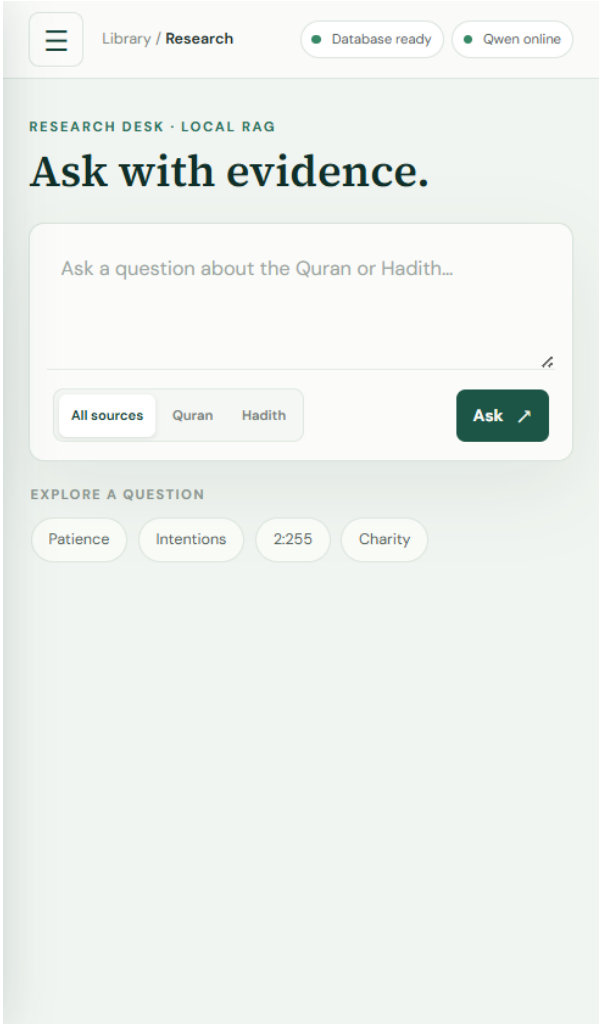
```
Project AL Majarrat/
  project_config.py       shared paths and model defaults
  src/                    application, extraction, indexing, validation
  data/corpora/           source/master CSV files
  data/db/                SQLite database
  data/datasets/          fine-tuning JSONL outputs
  data/raw/               raw data area
  data/models/            model artifacts area
  tests/                  project health tests
  web/                    browser interface
  docs/                   project documentation
```

5.5 Operational User Interfaces

CLI examples: python src/al_majarrat.py --ask "What does the Quran say about patience?"; python src/al_majarrat.py --test-retrieval "2:255"; python src/search_majarrat.py. The web app defaults to 127.0.0.1:8765 and supports all/Quran/Hadith filters, suggested queries, answer copying, saved conversations, and per-conversation deletion. History is browser-local and is not synchronized to another device.

5.6 User Interface Screenshots

Figure 5.1 — Browser interface in its responsive mobile layout.


Library / Research
Database ready
Qwen online
RESEARCH DESK · LOCAL RAG
Ask with evidence.
Ask a question about the Quran or Hadith...
All sources
Quran
Hadith
Ask ↗
EXPLORE A QUESTION
Patience
Intentions
2:255
Charity

Chapter 6 — Results and Discussion

6.1 Verified Database Snapshot

Counts below were read from the populated SQLite database during report preparation. They describe stored rows and vector coverage, not answer accuracy.

Measure	Count	Interpretation
Hadith source rows	34,157	Six loaded collections.
Quran database rows	56,124	Arabic and translation rows; not unique verses.
Distinct Quran verse keys	6,236	Unique verse references in this database.
Hadith vector rows	34,157	Count matches Hadith source row total.
Quran vector rows	6,236	Count matches unique Quran verse keys.
FTS Hadith rows	34,157	External-content index rows.
FTS Quran rows	56,124	External-content index rows.

Hadith collection	Rows
Sahih al-Bukhari	7,580
Sahih Muslim	7,360
Sunan an-Nasa'i	5,679
Sunan Abi Dawud	5,272
Jami' at-Tirmidhi	3,926
Sunan Ibn Majah	4,340
Total	34,157

6.2 Functional Verification

Check	Result	Scope / caveat
Project health suite	4 tests passed	Paths, data files, missing DB, valid schema.
Corpus presence	Passed	All configured source files present.
SQLite readiness	Passed	Required tables; 34,157 Hadith and 56,124 Quran rows.
Ollama validation	Skipped	--skip-ollama; live generation not certified.
Conversation flow	Implemented/tested	Browser smoke test reopened, reloaded, and deleted history.
Responsive UI	Implemented/tested	Tablet/mobile sidebar open and close checked.
Retrieval benchmark	Not performed	No labeled query set or accuracy metric.
User study / survey	Not performed	No user-study results claimed.

6.3 Example Input and Expected System Path

Example input: "2:255". The direct Quran reference path resolves a verse key and retrieves a preferred non-empty English translation. The application formats the reference and source text for display; if generation is enabled, the local model receives the evidence and is instructed to cite its reference. This describes the implemented path, not an evaluation of a particular answer.

6.4 Analysis of Results

The current database has vector coverage for all Hadith rows and distinct Quran verse keys in the snapshot. Health tests show that core path/data/schema checks behave as expected. These results confirm readiness of important local components but are not a retrieval-quality benchmark.

A full evaluation should measure Recall@k/MRR on expert-labeled references, citation correctness, unsupported-claim rate, response latency, and usability. The four existing unit tests do not measure these properties. The live Ollama service was not exercised in the recorded validation run, so generation quality and live model connectivity remain environment-dependent.

Chapter 7 — Conclusion and Future Scope

7.1 Summary of Work

Al-Majarrat provides a local research workflow over Quran and Hadith corpora. It combines corpus processing, populated SQLite/FTS5 tables, vector indexes, direct reference lookup, hybrid retrieval with RRF, prompt-constrained local generation, CLI and web interfaces, health validation, browser history, and a responsive UI. Code, data, tests, web assets, and docs are separated.

7.2 Achievements

- Prepared a database with 34,157 Hadith records and 56,124 Quran rows.
- Populated vectors for 34,157 Hadiths and 6,236 distinct Quran verse keys.
- Implemented lexical and semantic retrieval with reciprocal-rank fusion.
- Added direct Quran reference and Surah handling.
- Implemented a local web UI/API and CLI.
- Added browser conversation persistence, reopen, and individual deletion.
- Passed all four project health tests and configured corpus/database checks.

7.3 Limitations

8. Generated answers may be wrong or overstate evidence; prompt rules cannot guarantee factuality.
9. No expert-labeled retrieval benchmark, factuality evaluation, performance study, or user survey was conducted.
10. The live Ollama chat endpoint was not checked in the recorded health run.
11. Translation, edition, grading, provenance, and license details should be verified before scholarly use or redistribution.
12. Browser history lives in one browser's localStorage; clearing browser data removes it.
13. The RAG workflow currently targets English answers; Arabic-first interaction is not its documented focus.
14. Vector storage/query scaling and latency have not been profiled.
15. The training utilities do not automatically load their adapter into default inference.

7.4 Future Enhancements

- Build an expert-reviewed evaluation set; report Recall@k, MRR, citation precision, and unsupported claims.
- Record source provenance, edition/version metadata, license notices, and refresh procedures.
- Evaluate retrieval filters, rank fusion, reranking, and translation-specific indexing.
- Add a model readiness/smoke-test workflow and reproducible runtime benchmarks.
- Add history export/import, retention controls, and privacy documentation.
- Improve deployment hardening, logging, error reporting, and operational guidance before non-local exposure.
- Profile response latency and memory on defined hardware.
- If fine-tuning is pursued, add adapter loading and held-out evaluation.

Chapter 8 — References

Technical references reviewed for this report (accessed 4 October 2026):

[1] SQLite, “FTS5 Extension.” https://www.sqlite.org/fts5.html

[2] Ollama, “OpenAI compatibility.” https://docs.ollama.com/api/openai-compatibility

[3] Ollama, “Embeddings.” https://docs.ollama.com/capabilities/embeddings

[4] Quran Foundation, Content API documentation. https://api-docs.quran.foundation/

[5] fawazahmed0, “Hadith API” dataset repository. https://github.com/fawazahmed0/hadith-api

[6] Qwen, Qwen2.5-3B-Instruct model card. https://huggingface.co/Qwen/Qwen2.5-3B-Instruct

[7] Python Software Foundation, “sqlite3.” https://docs.python.org/3/library/sqlite3.html

[8] Project files reviewed: project_config.py; src/al_majarrat.py; corpus, index, search, web, validation and training scripts; tests/test_project_health.py.

[9] Al-Majarrat README, setup and operational instructions.

Dataset/API terms and model licenses may vary by source or change over time. Verify current terms, attribution, and translation permissions before publishing or redistributing corpus data.

Chapter 9 — Appendix

9.1 Database Structure Summary

```
hadiths(id, collection_slug, collection_name_en/ar, book_id/name, chapter_id/name,
        hadith_id, hadith_id_in_book, narrator, text_english, text_arabic, grading, reference)
quran_verses(id, chapter_number, verse_number, verse_key, text_uthmani,
             translation_resource_id, translation_name, translation_author, translation_text)
hadiths_fts: FTS5 external-content index -> hadiths.id
quran_fts: FTS5 external-content index -> quran_verses.id
hadith_vector_index(id, hadith_db_id UNIQUE, reference, embedding_json)
quran_vector_index(id, verse_key UNIQUE, embedding_json)
```

9.2 Local API Summary

Method / route	Purpose
GET /	Serve web/index.html.
GET /api/status	Return database counts and best-effort Ollama availability.
POST /api/ask	Accept {"query":"...", "mode":"all|quran|hadith"} and return answer/sources.

POST /api/ask enforces a non-empty query, a 2,000-character query limit, allowed source modes, and a 16 KB request body limit. The server defaults to 127.0.0.1.

9.3 Validation and Test Evidence

```
python src/validate_project.py --skip-ollama
python -m unittest tests/test_project_health.py
```

Recorded result: source checks and database validation passed; the validator reported 34,157 Hadith rows and 56,124 Quran rows; all four unittest cases passed. Ollama was skipped, so this does not certify a live model request.

9.4 Main Project Files

Path	Description
project_config.py	Shared paths, DB path, model endpoints, preferred translation.
src/al_majarrat.py	Hybrid RAG, prompt composition, local generation, CLI.
src/create_master_corpus.py	Master corpus and SQLite/FTS builder.
src/build_vector_index.py	Embedding generation and persistence.
src/search_majarrat.py	Lexical Quran/Hadith CLI search.
src/serve_web_ui.py	Local HTTP server, status endpoint, ask API.
web/index.html	Responsive UI and browser-local conversation history.
src/validate_project.py and tests/	Health checks and unit tests.
src/export_finetuning_dataset.py; src/train_qwen_lora.py	Dataset export and optional QLoRA utilities.

9.5 User Operation Quick Reference

```
Start web UI:       python src/serve_web_ui.py
Ask a question:     python src/al_majarrat.py --ask "What does the Quran say about patience?"
Test retrieval:     python src/al_majarrat.py --test-retrieval "2:255"
Lexical search:     python src/search_majarrat.py
```

```
Validate data/DB:   python src/validate_project.py --skip-ollama
Full model check:   python src/validate_project.py
```